

Raport Końcowy: System Wykrywania Phishingu w Czasie Rzeczywistym

1. Wstęp

Projekt ma na celu identyfikację potencjalnie złośliwych domen rejestrowanych na bieżąco, aby chronić znane marki (np. PayPal, Apple, Google) przed atakami phishingowymi, a w szczególności przed tzw. *typosquattingiem*. System nasłuchuje w czasie rzeczywistym logów Certificate Transparency (CT), przetwarza nazwy domen, a następnie poddaje je podwójnej klasyfikacji: heurystycznej oraz opartej na uczeniu maszynowym.

2. Architektura Systemu

Cały system został podzielony na kilka kluczowych modułów:

- **Ingestia Danych:** System wykorzystuje bibliotekę `certstream`, aby otrzymywać powiadomienia o nowo wystawianych certyfikatach SSL/TLS na świecie. Dla zachowania stabilności zaimplementowano również metody zapasowe z wykorzystaniem `UrlScan.io` oraz `MockCertstream` dla celów testowych.
- **Analiza Heurystyczna:** System generuje wszystkie możliwe kombinacje typosquattingowe dla chronionych marek z wykorzystaniem logiki `dnstwist` oraz analizy wyrażeń regularnych do wychwytywania zakamuflowanych nazw wewnątrz długich domen.
- **Uczenie Maszynowe (ML):** Do detekcji bardziej skomplikowanych wariantów wyuczono model *Random Forest*. Wyciąga on cztery główne cechy z domeny:
 - Odległość Levenshteina do chronionych marek
 - Entropię Shannona
 - Długość domeny
 - Liczbę nietypowych znaków specjalnych
- **Threat Intelligence:** By unikać fałszywych alarmów (False Positives), klasyfikacje wspierane są przez wywołania do serwisów *VirusTotal* oraz *Google Safe Browsing API* (wykonywane asynchronicznie, by nie blokować przepływu danych).

3. Kroki do odtworzenia projektu (Replication Steps)

Aby w pełni zreplikować projekt, wdrożyć aplikację na nowym urządzeniu i uruchomić proces detekcji, należy wykonać następujące kroki:

Krok 1: Klonowanie repozytorium i konfiguracja

Pobierz kod źródłowy projektu. Następnie zainstaluj wymagane pakiety w wirtualnym środowisku:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

W głównym katalogu projektu (`inference_app/src/`) utwórz plik `.env` wzorując się na dostępnym kodzie, dodając klucze API niezbędne do weryfikacji zagrożeń:

```
VT_API_KEY=twoj_klucz_virustotal  
GSB_API_KEY=twoj_klucz_google_safe_browsing
```

Krok 2: Wygenerowanie zestawów danych i trening modelu

Aplikacja treningowa została zaprojektowana tak, aby potrafiła samoczynnie pozyskać aktualne dane ze źródeł otwartych (OpenPhish dla danych złośliwych oraz Tranco Top 1M dla danych łagodnych). Należy uruchomić skrypt treningowy przekazując flagę środowiska publicznego:

```
python training_app/src/train.py --dataset public
```

Skrypt wyekstrahuje zbiór cech, zbalansuje wagi, wytrenuje klasyfikator `RandomForestClassifier` i wygeneruje w folderze `shared/models/` plik `model_weights.pkl`.

Krok 3: Uruchomienie Nasłuchu Głównego (Inference)

Po zbudowaniu modelu, system jest gotowy na przyjmowanie realnego ruchu:

```
python inference_app/src/main.py
```

Aplikacja natychmiast rozpocznie asynchroniczny nasłuch strumienia *CertStream*. Logi dotyczące wykrytych domen (heurystyka, ML oraz Threat Intel) będą zapisywane strumieniowo do pliku `results/phishing_results.csv`.

Możliwe jest podanie flagi `--mock`, która zamarkuje ruch lokalnym generatorem celem weryfikacji.

Krok 4: Ewaluacja działania

Aby ocenić skuteczność aktualnego modelu po zebraniu danych z produkcji, można uruchomić automatyczny audyt zebranych klasyfikacji:

```
python training_app/src/evaluate.py
```

Wynikiem działania będzie wyświetlenie parametrów: *Accuracy*, *Precision*, *Recall* oraz *F1-Score*.

4. Wnioski

Rozwiązanie z powodzeniem przetwarza duże potoki danych z Certificate Transparency. Integracja klasycznej heurystyki z elementami uczenia maszynowego oraz Threat Intelligence pozwala uzyskać wysoką precyzję, redukując jednocześnie odsetek fałszywych alarmów wynikających z podobieństw leksykalnych domen nienależących do phishingowych serwerów.